

# Java and Spring Boot Interview Notes

İbrahim Utku Adanur

September 25, 2026

## Contents

|          |                                                                                                   |          |
|----------|---------------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Java Interview Questions</b>                                                                   | <b>3</b> |
| 1.1      | What is the Difference Between <code>==</code> and <code>equals()</code> ? . . . . .              | 3        |
| 1.2      | What Does the <code>final</code> Keyword Mean for Variables, Methods, and Classes? . . . . .      | 3        |
| 1.3      | What is the <code>@Qualifier</code> Annotation in Spring? . . . . .                               | 3        |
| 1.4      | Can a Static Method Be Overridden in Java? . . . . .                                              | 4        |
| 1.5      | Which Memory Area is Cleaned by the Garbage Collector: Heap or Stack? . . . . .                   | 4        |
| 1.6      | Why is <code>String</code> Immutable in Java? . . . . .                                           | 5        |
| 1.7      | What is the Difference Between <code>HashSet</code> and <code>TreeSet</code> ? . . . . .          | 6        |
| 1.8      | What is the Difference Between <code>HashMap</code> and <code>HashSet</code> ? . . . . .          | 6        |
| 1.9      | What is a Bean? . . . . .                                                                         | 6        |
| 1.10     | What is the <code>ApplicationContext</code> ? . . . . .                                           | 7        |
| 1.11     | What Does <code>@SpringBootApplication</code> Do Internally? . . . . .                            | 7        |
| 1.12     | What is <code>@Autowired</code> in Spring? . . . . .                                              | 8        |
| 1.13     | What is a Singleton? . . . . .                                                                    | 8        |
| 1.14     | Are Singleton Beans Thread-Safe in Spring? . . . . .                                              | 9        |
| 1.15     | What is the Difference Between Abstraction and Encapsulation? . . . . .                           | 10       |
| 1.16     | What is the Single Responsibility Principle? . . . . .                                            | 10       |
| 1.17     | What is the Open/Closed Principle? . . . . .                                                      | 10       |
| 1.18     | What is the Liskov Substitution Principle? . . . . .                                              | 10       |
| 1.19     | What is the Interface Segregation Principle? . . . . .                                            | 11       |
| 1.20     | What is the Dependency Inversion Principle? . . . . .                                             | 12       |
| 1.21     | How Can Objects Be Created Without Using the <code>new</code> Keyword? . . . . .                  | 13       |
| 1.22     | What are the Different ClassLoaders in the JVM? . . . . .                                         | 14       |
| 1.23     | What is the Difference Between Heap and Stack Memory in Java? . . . . .                           | 14       |
| 1.24     | What is the Difference Between an Exception and an Error in Java? . . . . .                       | 15       |
| 1.25     | What is the Purpose of the <code>finally</code> Block in Java? . . . . .                          | 15       |
| 1.26     | Which Collection Maintains Insertion Order and Avoids Duplicates? . . . . .                       | 15       |
| 1.27     | How Does <code>ConcurrentHashMap</code> Handle Concurrency? . . . . .                             | 16       |
| 1.28     | What is the Difference Between Shallow Copy and Deep Copy? . . . . .                              | 17       |
| 1.29     | What is a Functional Interface in Java? . . . . .                                                 | 17       |
| 1.30     | How Can You Create Your Own Functional Interface in Java? . . . . .                               | 19       |
| 1.31     | What is the Purpose of <code>Optional</code> in Java? . . . . .                                   | 19       |
| 1.32     | What is the Difference Between Intermediate and Terminal Operations in<br>Java Streams? . . . . . | 20       |
| 1.33     | How Can You Access Properties from <code>application.properties</code> in Spring? . . . . .       | 20       |

|      |                                                                              |    |
|------|------------------------------------------------------------------------------|----|
| 1.34 | What is the Purpose of Spring Profiles? . . . . .                            | 21 |
| 1.35 | What is the Difference Between JPA and Hibernate? . . . . .                  | 22 |
| 1.36 | How Do You Perform CRUD Operations Using JPA? . . . . .                      | 22 |
| 1.37 | How Do You Implement Pagination in JPA? . . . . .                            | 23 |
| 1.38 | What Are the Different Identifier Generation Strategies in JPA? . . . . .    | 23 |
| 1.39 | What Fetch Strategies Are Available in JPA? . . . . .                        | 24 |
| 1.40 | What Caching Mechanisms Are Supported by JPA? . . . . .                      | 25 |
| 1.41 | What is a Composite Key in SQL? . . . . .                                    | 25 |
| 1.42 | When Do We Use an OUTER JOIN in SQL? . . . . .                               | 26 |
| 1.43 | What is a Database View and When Do We Use It? . . . . .                     | 26 |
| 1.44 | What is the Difference Between a Process and a Thread? . . . . .             | 27 |
| 1.45 | What is Multithreading? . . . . .                                            | 27 |
| 1.46 | What is Multiprocessing? . . . . .                                           | 28 |
| 1.47 | What is the Difference Between Multithreading and Multiprocessing? . . . . . | 28 |
| 1.48 | What is a Race Condition? . . . . .                                          | 29 |
| 1.49 | What is Synchronization and Why is it Needed? . . . . .                      | 29 |
| 1.50 | What is a Deadlock? . . . . .                                                | 30 |
| 1.51 | What Does Thread-Safe Mean? . . . . .                                        | 30 |
| 1.52 | What Are the Advantages of Stored Procedures? . . . . .                      | 30 |

# 1 Java Interview Questions

## 1.1 What is the Difference Between == and equals()?

In Java, `==` compares primitive values directly. For objects, it checks whether two references point to the same object. The `equals()` method is used for logical equality and can be overridden by a class to compare object contents. The default implementation of `equals()` in the `Object` class performs reference comparison.

### Key Point

- For primitives, `==` compares values.
- For objects, `==` compares references.
- `equals()` is used for logical or content equality.

## 1.2 What Does the final Keyword Mean for Variables, Methods, and Classes?

In Java, the `final` keyword prevents further modification depending on where it is used. A final variable cannot be reassigned, a final method cannot be overridden and a final class cannot be extended. For object references, `final` prevents reassignment of the reference, but it does not make the object itself immutable.

## 1.3 What is the @Qualifier Annotation in Spring?

The `@Qualifier` annotation is used when multiple beans of the same type exist in the Spring `ApplicationContext` and Spring cannot determine which one should be injected.

### @Qualifier vs @Primary

`@Primary` defines the default bean when multiple beans of the same type exist.

`@Qualifier` explicitly selects a specific bean.

`@Primary` -> preferred/default bean

`@Qualifier` -> explicitly selected bean

### Important Note

Using `@Qualifier` binds that dependency to a specific implementation.

Therefore, it is suitable when the implementation is known when the application is configured.

If the implementation must be selected dynamically at runtime, for example based on a user's payment method, a Strategy or Factory-based design is usually more appropriate.

## Interview Answer

The `@Qualifier` annotation is used when multiple beans of the same type exist in the Spring context. It tells Spring exactly which bean should be injected and resolves ambiguity during dependency injection.

It is mainly used for static dependency selection. If the implementation must be chosen dynamically at runtime, a Strategy or Factory approach can be used instead.

## 1.4 Can a Static Method Be Overridden in Java?

No. Static methods cannot be overridden in Java.

If a subclass defines a static method with the same signature as a static method in its superclass, this is called *method hiding*, not method overriding.

### Method Overriding vs Method Hiding

Instance method:

- > Can be overridden
- > Runtime polymorphism
- > Actual object type determines the method

Static method:

- > Cannot be overridden
- > Can be hidden
- > Reference/class type determines the method

## Interview Answer

Static methods cannot be overridden in Java because they belong to the class rather than an instance. If a subclass defines a static method with the same signature, it hides the superclass method. This is called method hiding. Static method calls are resolved based on the reference or class type rather than the actual object type.

## 1.5 Which Memory Area is Cleaned by the Garbage Collector: Heap or Stack?

The Java Garbage Collector primarily manages **heap memory**.

Objects created with **new** are generally allocated on the heap.

For example:

```
Person person = new Person("Utku");
```

The local reference **person** may exist in the current stack frame, while the actual **Person** object is stored on the heap.

If the object becomes unreachable, it becomes eligible for garbage collection.

```
Person person = new Person("Utku");
```

```
person = null;
```

After the reference is removed, if no other reachable reference points to the object, the object may be reclaimed by the Garbage Collector.

Being eligible for garbage collection does not mean that the object is immediately removed. The JVM decides when garbage collection should run.

### What Happens to Stack Memory?

Stack memory is managed automatically using stack frames.

Each method call creates a stack frame that contains information such as local variables and references.

```
void calculate() {  
    int x = 10;  
    int y = 20;  
}
```

When the method finishes, its stack frame is automatically removed. Therefore, the Garbage Collector does not normally clean stack memory.

Heap:

- > Stores objects
- > Managed mainly by the Garbage Collector
- > Unreachable objects may be reclaimed

Stack:

- > Stores method frames and local variables
- > Automatically managed
- > Frames are removed when methods return

### Interview Answer

The Java Garbage Collector mainly manages heap memory. Objects are allocated on the heap and when they are no longer reachable, they become eligible for garbage collection. Stack memory is managed automatically through stack frames, which are removed when methods return.

## 1.6 Why is String Immutable in Java?

A `String` object cannot be modified after it has been created. Operations that appear to modify a `String` actually create a new `String` object.

### Interview Answer

`String` is immutable in Java mainly for security, safe `String` Pool sharing, thread safety, and reliable hashing. Since a `String` cannot change after creation, it can safely be shared between threads, reused from the `String` Pool, and used as a stable key in hash-based collections such as `HashMap`.

## 1.7 What is the Difference Between HashSet and TreeSet?

Both `HashSet` and `TreeSet` implement the `Set` interface, meaning that they do not allow duplicate elements.

`HashSet`:

- > Unique elements
- > No guaranteed ordering
- > Hash-based
- > Uses `hashCode()` and `equals()`
- > Average  $O(1)$  operations

`TreeSet`:

- > Unique elements
- > Sorted elements
- > Tree-based
- > Uses `Comparable` or `Comparator`
- >  $O(\log n)$  operations

### Interview Answer

Both `HashSet` and `TreeSet` store unique elements. `HashSet` uses hashing and does not guarantee ordering, while `TreeSet` keeps elements sorted using a tree structure. `HashSet` provides average  $O(1)$  operations, whereas `TreeSet` provides  $O(\log n)$  operations. `HashSet` relies mainly on `hashCode()` and `equals()`, while `TreeSet` relies on `Comparable` or `Comparator`.

## 1.8 What is the Difference Between HashMap and HashSet?

`HashMap` stores data as key-value pairs, while `HashSet` stores only unique elements.

### Interview Answer

`HashMap` stores key-value pairs, while `HashSet` stores only unique elements. `HashMap` does not allow duplicate keys, although duplicate values are allowed. `HashSet` does not allow duplicate elements. Internally, `HashSet` is backed by a `HashMap`, where the set elements are stored as map keys with a dummy value.

## 1.9 What is a Bean?

A bean is simply an object whose lifecycle is managed by Spring.

Spring may create beans automatically by discovering classes annotated with annotations such as:

```
@Component
@Service
@Repository
@Controller
@RestController
```

Beans can also be explicitly declared using the `@Bean` annotation.

## 1.10 What is the ApplicationContext?

The `ApplicationContext` is the central Spring IoC container.

It is responsible for creating, storing, configuring, and connecting Spring-managed objects, which are called **beans**.

`ApplicationContext` can be thought of as the container in which Spring-managed objects live.

## 1.11 What Does @SpringBootApplication Do Internally?

`@SpringBootApplication` is a convenience annotation that combines:

```
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan
```

`@SpringBootConfiguration` is based on `@Configuration` and allows Spring configuration and bean definitions.

For example:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentClient paymentClient() {
        return new PaymentClient();
    }
}
```

The object returned by a `@Bean` method is registered in the Spring `ApplicationContext`.

`@ComponentScan` searches the application's package and its subpackages for classes such as:

```
@Component
@Service
@Repository
@Controller
@RestController
```

and registers them as Spring beans.

`@EnableAutoConfiguration` automatically configures the application based on the libraries available on the classpath, application properties and existing beans.

For example, if `spring-boot-starter-web` is present, Spring Boot automatically configures much of the Spring MVC infrastructure.

Finally:

```
SpringApplication.run(DemoApplication.class, args);
```

starts the application and initializes the Spring `ApplicationContext`.

## Interview Answer

`@SpringBootApplication` combines configuration, component scanning and auto-configuration. It allows Spring to discover and create beans, configure the application automatically and manage those beans inside the `ApplicationContext`. The `ApplicationContext` is the IoC container responsible for managing beans and injecting their dependencies.

### 1.12 What is @Autowired in Spring?

`@Autowired` is used for dependency injection. It tells Spring to find a matching bean from the `ApplicationContext` and inject it.

`@Service`

```
public class OrderService {  
  
    private final PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

Since Spring 4.3, if a class has only one constructor, `@Autowired` is not required because Spring automatically uses that constructor for dependency injection.

If a class has multiple constructors, `@Autowired` can be used to indicate which constructor Spring should use.

`final` is commonly used with constructor injection because the dependency is assigned once during object creation and cannot later be reassigned to another object.

`@Autowired` can also be used for setter and field injection, but constructor injection is generally preferred.

## Interview Answer

`@Autowired` tells Spring to inject a matching bean from the `ApplicationContext`. With a single constructor, it is usually not needed. With multiple constructors, it can specify which constructor should be used for injection. Constructor-injected dependencies are often declared `final` so their references cannot be reassigned.

### 1.13 What is a Singleton?

A singleton is a design pattern that ensures only one instance of a class exists and provides a shared access point to that instance.

In Spring, the default bean scope is `singleton`. This means Spring creates one bean instance per `ApplicationContext` and reuses that same instance whenever the bean is injected.

For example:

```
@Service  
public class OrderService {  
}
```

If `OrderService` is injected into multiple classes, Spring normally provides the same bean instance.

```
ApplicationContext
|
+-- one OrderService instance
    |
    +-- used by Controller A
    +-- used by Service B
```

A Spring singleton is not automatically thread-safe. If the bean contains mutable shared state, multiple threads may access and modify the same data.

### Interview Answer

A singleton means that only one instance of an object is used within a given context. In Spring, singleton is the default bean scope, so one bean instance is created per `ApplicationContext` and shared wherever that bean is injected. However, singleton does not automatically mean thread-safe.

## 1.14 Are Singleton Beans Thread-Safe in Spring?

No. A singleton bean is not automatically thread-safe.

By default, Spring creates one bean instance per `ApplicationContext`. Multiple threads may use the same bean instance at the same time.

A stateless singleton bean is generally safe:

```
@Service
public class OrderService {

    public int calculateTotal(int price, int quantity) {
        return price * quantity;
    }
}
```

However, mutable shared state may cause race conditions:

```
@Service
public class CounterService {

    private int counter = 0;

    public int increment() {
        return ++counter;
    }
}
```

Multiple threads can modify `counter` concurrently.

## Interview Answer

Spring singleton beans are not automatically thread-safe. A singleton means that one bean instance is shared within the `ApplicationContext`. Stateless beans are generally safe, but mutable shared state can cause race conditions and must be handled carefully.

### 1.15 What is the Difference Between Abstraction and Encapsulation?

#### Interview Answer

Abstraction hides implementation details and focuses on what an object does. Encapsulation protects an object's internal state by controlling access to its data. Abstraction is about hiding complexity, while encapsulation is about restricting direct access to data.

### 1.16 What is the Single Responsibility Principle?

#### Interview Answer

The Single Responsibility Principle states that a class should have one main responsibility and only one reason to change. Unrelated responsibilities should be separated into different classes.

### 1.17 What is the Open/Closed Principle?

#### Interview Answer

The Open/Closed Principle states that software components should be open for extension but closed for modification. New behavior should preferably be added through new implementations instead of repeatedly changing existing code.

### 1.18 What is the Liskov Substitution Principle?

The Liskov Substitution Principle states that a subclass should be usable wherever its parent class is expected without breaking the expected behavior.

```
class Vehicle {
    public void startEngine() {
        System.out.println("Engine_started");
    }
}

class Bicycle extends Vehicle {

    @Override
    public void startEngine() {
        throw new UnsupportedOperationException();
    }
}
```

A `Bicycle` cannot safely replace a `Vehicle` because it cannot provide the expected `startEngine()` behavior.

## Interview Answer

A subclass should be replaceable with its parent type without breaking the expected behavior of the program.

### 1.19 What is the Interface Segregation Principle?

The Interface Segregation Principle states that a class should not be forced to implement methods it does not need.

For example:

```
interface Worker {  
    void work();  
    void eat();  
}  
  
class Robot implements Worker {  
  
    public void work() {  
        System.out.println("Working");  
    }  
  
    public void eat() {  
        // Robot does not need this method  
    }  
}
```

A better design uses smaller and more focused interfaces:

```
interface Workable {  
    void work();  
}  
  
interface Eatable {  
    void eat();  
}  
  
class Robot implements Workable {  
    public void work() {  
    }  
}  
  
class Human implements Workable, Eatable {  
    public void work() {  
    }  
  
    public void eat() {  
    }  
}
```

## Interview Answer

The Interface Segregation Principle states that classes should not be forced to depend on methods they do not use. Small and focused interfaces are preferred over large general-purpose interfaces.

### 1.20 What is the Dependency Inversion Principle?

The Dependency Inversion Principle states that high-level classes should depend on abstractions rather than concrete implementations.

For example, this creates tight coupling:

```
class OrderService {  
  
    private CreditCardPaymentService paymentService =  
        new CreditCardPaymentService();  
  
    public void checkout() {  
        paymentService.pay();  
    }  
}
```

A better design depends on an abstraction:

```
interface PaymentService {  
    void pay();  
}  
  
class CreditCardPaymentService  
    implements PaymentService {  
  
    public void pay() {  
        System.out.println("Paid_by_credit_card");  
    }  
}  
  
class OrderService {  
  
    private final PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
  
    public void checkout() {  
        paymentService.pay();  
    }  
}
```

Now `OrderService` depends on the `PaymentService` abstraction instead of a specific payment implementation.

## Interview Answer

The Dependency Inversion Principle means that high-level classes should depend on abstractions rather than concrete implementations. For example, `OrderService` should depend on `PaymentService`, not directly on `CreditCardPaymentService`.

## 1.21 How Can Objects Be Created Without Using the new Keyword?

Apart from directly using the `new` keyword, Java objects can be created or obtained in several ways.

### Reflection

Reflection can invoke a constructor dynamically at runtime.

```
User user = User.class.getDeclaredConstructor().newInstance();
```

The older `Class.newInstance()` method is deprecated. Modern Java prefers `getDeclaredConstructor`.

### Cloning

An object can be created as a copy of another object using `clone()`.

```
User user2 = (User) user1.clone();
```

The class normally needs to implement `Cloneable`.

### Deserialization

A previously serialized object can be reconstructed from a stream.

```
ObjectInputStream in =
    new ObjectInputStream(
        new FileInputStream("user.ser")
    );
```

```
User user = (User) in.readObject();
```

### Factory Methods

Objects can also be obtained through factory methods:

```
Integer number = Integer.valueOf(10);
```

A factory method may internally create a new object or return an existing one.

## Interview Answer

Apart from using the `new` keyword directly, objects can be created using reflection, cloning, and deserialization. Objects can also be obtained through factory methods or frameworks such as Spring.

## 1.22 What are the Different ClassLoaders in the JVM?

A `ClassLoader` is responsible for loading Java classes into the JVM.

The main built-in class loaders are:

`Bootstrap ClassLoader`  
`Platform ClassLoader`  
`Application ClassLoader`

### Bootstrap ClassLoader

The `Bootstrap ClassLoader` loads core Java classes such as `java.lang.String` and `java.lang.Object`. It is part of the JVM itself.

### Platform ClassLoader

The `Platform ClassLoader` loads Java platform-level classes.

In older Java versions, a similar role was performed by the `Extension ClassLoader`.

### Application ClassLoader

The `Application ClassLoader` loads application classes and dependencies available on the classpath.

For example, classes such as:

`OrderService`  
`JobApplicationController`  
`PaymentService`

are normally loaded by the `Application ClassLoader`.

Class loaders usually follow the **parent delegation model**, where a class loader first asks its parent to load the requested class.

### Interview Answer

The JVM mainly uses `Bootstrap`, `Platform`, and `Application ClassLoaders`. `Bootstrap` loads core Java classes, `Platform` loads platform-level classes, and `Application` loads application classes and classpath dependencies. They normally follow the parent delegation model.

## 1.23 What is the Difference Between Heap and Stack Memory in Java?

### Interview Answer

Stack memory stores method frames and local variables, while heap memory stores objects. Each thread has its own stack, whereas the heap is shared. Stack frames are removed automatically when methods return, while heap objects are managed by the Garbage Collector.

## 1.24 What is the Difference Between an Exception and an Error in Java?

Both Exception and Error inherit from Throwable.

An Exception usually represents a problem that an application can anticipate and potentially handle.

Examples include:

```
IOException  
SQLException  
NullPointerException  
IllegalArgumentException
```

An Error represents a more serious JVM or system-level problem.

Examples include:

```
OutOfMemoryError  
StackOverflowError
```

Although an Error can technically be caught, applications normally should not try to recover from such serious failures.

### Interview Answer

Exceptions usually represent application-level problems that can potentially be handled, while Errors represent serious JVM or system-level failures that applications normally should not try to recover from.

## 1.25 What is the Purpose of the finally Block in Java?

The finally block is used for code that should execute whether an exception occurs or not.

It is commonly used for cleanup operations such as releasing resources.

The finally block normally executes regardless of whether the try block succeeds or throws an exception.

In modern Java, try-with-resources is often preferred for closing resources automatically.

### Interview Answer

The finally block is used for code that should run whether an exception occurs or not, typically for cleanup and resource release.

## 1.26 Which Collection Maintains Insertion Order and Avoids Duplicates?

LinkedHashSet maintains insertion order while also preventing duplicate elements.

```
Set<String> names = new LinkedHashSet<>();
```

```
names.add("Utku");  
names.add("Ali");  
names.add("Utku");  
names.add("Mehmet");
```

```
System.out.println(names);
```

The output is:

```
[Utku, Ali, Mehmet]
```

The second "Utku" is ignored because sets do not allow duplicates, while the insertion order is preserved.

### Interview Answer

`LinkedHashSet` is used when unique elements are required while preserving insertion order.

## 1.27 How Does `ConcurrentHashMap` Handle Concurrency?

`ConcurrentHashMap` provides thread-safe access while allowing multiple threads to operate on the map concurrently.

Unlike a fully synchronized map, it does not lock the entire map for every operation. In modern Java:

- Read operations such as `get()` are mostly non-blocking.
- Write operations use fine-grained synchronization and CAS.
- Different threads can update different buckets concurrently.

For example:

```
ConcurrentHashMap<String, Integer> map =  
    new ConcurrentHashMap<>();
```

```
map.put("A", 1);  
map.put("B", 2);
```

Two threads may work on different parts of the map at the same time without locking the whole structure.

`ConcurrentHashMap` does not allow `null` keys or values.

### Interview Answer

`ConcurrentHashMap` provides thread-safe access without locking the entire map. Reads are mostly non-blocking, while updates use fine-grained synchronization and CAS on individual buckets. This reduces contention and allows multiple threads to operate concurrently.

## 1.28 What is the Difference Between Shallow Copy and Deep Copy?

A **shallow copy** creates a new outer object, but nested object references are shared.

```
Person p1 =  
    new Person("Utku", new Address("Ankara"));
```

```
Person p2 =  
    new Person(p1.name, p1.address);
```

Both objects refer to the same **Address** object.

Therefore:

```
p2.address.city = "Istanbul";
```

also affects the address seen through **p1**.

A **deep copy** also creates separate copies of nested mutable objects.

```
Person p2 = new Person(  
    p1.name,  
    new Address(p1.address.city)  
);
```

Now the two **Person** objects have independent **Address** objects.

### Interview Answer

A shallow copy creates a new outer object but shares references to nested objects. A deep copy creates independent copies of both the outer object and its nested mutable objects.

## 1.29 What is a Functional Interface in Java?

A functional interface is an interface that contains exactly one abstract method.

Its main purpose is to represent a behavior that can be implemented using a lambda expression.

For example:

```
@FunctionalInterface  
interface Calculator {  
    int calculate(int a, int b);  
}
```

Without lambda expressions, we could create a separate implementation:

```
class AddCalculator implements Calculator {  
  
    @Override  
    public int calculate(int a, int b) {  
        return a + b;  
    }  
}
```

```
Calculator calculator = new AddCalculator();
```

We could also use an anonymous class:

```
Calculator calculator = new Calculator() {  
  
    @Override  
    public int calculate(int a, int b) {  
        return a + b;  
    }  
};
```

Here, the interface itself is not instantiated. Java creates an anonymous class that implements the interface and creates an instance of that class.

With a lambda expression, the same behavior can be written much more concisely:

```
Calculator calculator = (a, b) -> a + b;
```

The lambda expression provides the implementation of the single abstract method in the functional interface.

Functional interfaces also allow behavior to be passed as a method parameter:

```
public void process(  
    int a,  
    int b,  
    Calculator calculator  
) {  
    System.out.println(  
        calculator.calculate(a, b)  
    );  
}
```

```
process(10, 5, (a, b) -> a + b);  
process(10, 5, (a, b) -> a - b);  
process(10, 5, (a, b) -> a * b);
```

The method remains the same, but the behavior passed to it can change.

Common functional interfaces in Java include:

```
Predicate<T>      -> input -> boolean  
Function<T, R>   -> input -> output  
Consumer<T>      -> input -> no return value  
Supplier<T>      -> no input -> output
```

## Interview Answer

A functional interface is an interface with exactly one abstract method. It allows a behavior to be represented with a lambda expression and passed around as a value, for example as a method parameter. Without lambdas, the same behavior would normally require a separate implementation class or an anonymous class.

## 1.30 How Can You Create Your Own Functional Interface in Java?

A custom functional interface is created by defining an interface with exactly one abstract method.

```
@FunctionalInterface
public interface Calculator {

    int calculate(int a, int b);
}
```

It can then be implemented using a lambda expression:

```
Calculator add = (a, b) -> a + b;

int result = add.calculate(5, 3);
```

The `@FunctionalInterface` annotation is optional, but recommended. It allows the compiler to verify that the interface contains only one abstract method.

A functional interface may still contain `default` and `static` methods.

### Interview Answer

To create a custom functional interface, define an interface with exactly one abstract method. The `@FunctionalInterface` annotation is optional but recommended because it lets the compiler enforce this rule.

## 1.31 What is the Purpose of Optional in Java?

`Optional<T>` is a container that represents a value that may or may not be present.

It is commonly used as a return type when a method may not find a result.

```
Optional<User> optionalUser =
    repository.findById(id);
```

Instead of manually checking for `null`, methods such as `orElse()`, `orElseGet()`, and `orElseThrow()` can be used.

For example:

```
public User getUserById(Long id) {
    return repository.findById(id)
        .orElseThrow(() ->
            new ResourceNotFoundException(
                "User_not_found"
            )
        );
}
```

If the user exists, `orElseThrow()` returns the `User`. If the value is absent, it throws the specified exception.

## Interview Answer

`Optional` represents a value that may or may not be present. It is commonly used for return values to express absence explicitly and reduce manual null checks.

### 1.32 What is the Difference Between Intermediate and Terminal Operations in Java Streams?

Intermediate operations transform a stream and return another stream.

Examples include:

```
filter()
map()
sorted()
distinct()
limit()
```

Terminal operations complete the stream pipeline and produce a result or side effect.

Examples include:

```
toList()
forEach()
count()
reduce()
collect()
findFirst()
```

For example:

```
List<String> names = users.stream()
    .filter(user -> user.getAge() >= 18)
    .map(User::getName)
    .toList();
```

Here, `filter()` and `map()` are intermediate operations, while `toList()` is a terminal operation.

Intermediate operations are usually lazy and are executed when a terminal operation is called.

## Interview Answer

Intermediate operations return another stream and build the processing pipeline. Terminal operations finish the pipeline and produce the final result. Intermediate operations are generally lazy.

### 1.33 How Can You Access Properties from `application.properties` in Spring?

A property can be injected using the `@Value` annotation.

For example:

```
app.name=Job Tracker
```

```
@RestController
public class AppController {

    @Value("${app.name}")
    private String appName;
}
```

Spring reads `app.name` from the configuration and injects its value into `appName`.

For multiple related properties, `@ConfigurationProperties` is usually preferred because it groups configuration values into a dedicated class.

### Interview Answer

Properties can be accessed using `@Value`. For multiple related properties, `@ConfigurationProperties` provides a cleaner and more maintainable approach.

## 1.34 What is the Purpose of Spring Profiles?

Spring Profiles allow different configurations to be used for different environments such as development, testing, and production.

For example:

```
application-dev.properties
application-test.properties
application-prod.properties
```

A profile can be activated using:

```
spring.profiles.active=dev
```

Profiles can also control which beans are created:

```
@Configuration
@Profile("dev")
public class DevConfig {
}
```

This configuration is only loaded when the `dev` profile is active.

### Interview Answer

Spring Profiles allow environment-specific configuration. For example, development, testing, and production environments can use different database settings or beans without changing the application code.

### 1.35 What is the Difference Between JPA and Hibernate?

JPA is a specification that defines how Java objects should be persisted and mapped to relational databases.

Hibernate is an implementation of the JPA specification.

JPA           -> specification / contract  
Hibernate -> implementation

For example, annotations such as `@Entity`, `@Id`, and `@OneToMany` are defined by JPA, while Hibernate performs the actual ORM work and communicates with the database.

Spring Data JPA provides another abstraction on top of JPA and simplifies repository operations.

```
Spring Data JPA
  |
  v
  JPA
  |
  v
  Hibernate
  |
  v
  Database
```

#### Interview Answer

JPA is a specification for object-relational mapping in Java, while Hibernate is one of its implementations. JPA defines the standard API and rules, and Hibernate performs the actual persistence operations.

### 1.36 How Do You Perform CRUD Operations Using JPA?

In standard JPA, CRUD operations can be performed using `EntityManager`:

```
Create -> persist()
Read   -> find()
Update -> merge()
Delete -> remove()
```

For example:

```
entityManager.persist(user);

User user =
    entityManager.find(User.class, 1L);

user.setName("Utku");
entityManager.merge(user);

entityManager.remove(user);
```

With Spring Data JPA, these operations are usually simplified through `JpaRepository`:

```
public interface UserRepository
    extends JpaRepository<User , Long> {
}
```

Common methods include:

```
save()           -> Create or Update
findById()       -> Read
findAll()        -> Read all
deleteById()     -> Delete
```

### Interview Answer

In standard JPA, CRUD operations are performed with `EntityManager` using `persist`, `find`, `merge`, and `remove`. With Spring Data JPA, `JpaRepository` provides methods such as `save`, `findById`, `findAll`, and `deleteById`.

## 1.37 How Do You Implement Pagination in JPA?

With Spring Data JPA, pagination is commonly implemented using `Pageable` and `Page<T>`.

```
Pageable pageable = PageRequest.of(0, 10);
```

```
Page<User> users =
    userRepository.findAll(pageable);
```

Here, 0 is the page number and 10 is the page size.

The returned `Page` contains both the current page data and pagination metadata such as total pages and total elements.

Sorting can also be added:

```
Pageable pageable = PageRequest.of(
    0,
    10,
    Sort.by("name").ascending()
);
```

In standard JPA, pagination can also be implemented using `setFirstResult()` and `setMaxResults()`.

### Interview Answer

With Spring Data JPA, pagination is implemented using `Pageable`, usually created with `PageRequest`, and repository methods return a `Page<T>` containing both the records and pagination metadata.

## 1.38 What Are the Different Identifier Generation Strategies in JPA?

JPA can generate primary key values using the `@GeneratedValue` annotation.

Common strategies include:

IDENTITY  
SEQUENCE  
TABLE  
AUTO  
UUID

For example:

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

IDENTITY uses the database's identity or auto-increment mechanism.

SEQUENCE uses a database sequence.

TABLE uses a separate table to generate identifiers.

AUTO allows the JPA provider to choose an appropriate strategy.

Modern JPA versions also support UUID generation.

## Interview Answer

JPA supports identifier generation strategies such as IDENTITY, SEQUENCE, TABLE, AUTO, and UUID. The appropriate strategy depends on the database and application requirements.

## 1.39 What Fetch Strategies Are Available in JPA?

JPA provides two main fetch strategies: LAZY and EAGER.

Suppose a `User` has many orders:

```
@OneToMany(fetch = FetchType.LAZY)
private List<Order> orders;
```

With LAZY loading, retrieving the user does not immediately load the orders.

```
User user = userRepository.findById(1L)
    .orElseThrow();
```

```
user.getName();    // Orders are not needed yet
user.getOrders();  // Orders may now be loaded
```

With EAGER loading:

```
@OneToMany(fetch = FetchType.EAGER)
private List<Order> orders;
```

the related orders are loaded as part of loading the user, even if they are not immediately used.

Default fetch types are:

```
@OneToMany -> LAZY
@ManyToMany -> LAZY
@ManyToOne -> EAGER
@OneToOne -> EAGER
```

## Interview Answer

LAZY loading delays loading related entities until they are needed, while EAGER loading fetches them together with the main entity. LAZY loading can avoid retrieving unnecessary data and is often preferred for large relationships.

## 1.40 What Caching Mechanisms Are Supported by JPA?

JPA mainly provides two levels of caching.

### First-Level Cache

The first-level cache belongs to the persistence context and is associated with an `EntityManager`. It is automatic.

```
User u1 = entityManager.find(User.class, 1L);
User u2 = entityManager.find(User.class, 1L);
```

Within the same persistence context, the second lookup can reuse the already managed entity instead of querying the database again.

### Second-Level Cache

The second-level cache can be shared between multiple persistence contexts.

It is optional and depends on the JPA provider and its configuration.

#### First-Level Cache

- > `EntityManager` / Persistence Context scoped
- > Automatic

#### Second-Level Cache

- > Shared across persistence contexts
- > Optional and provider-dependent

Query caching is not a standard JPA cache mechanism and may be provided by implementations such as Hibernate.

## Interview Answer

JPA has a first-level cache associated with the persistence context, which is automatic. It can also use an optional second-level cache shared across multiple persistence contexts, depending on the JPA provider and configuration.

## 1.41 What is a Composite Key in SQL?

A composite key is a key made of two or more columns that together uniquely identify a row.

For example:

```
CREATE TABLE student_course (
    student_id BIGINT,
    course_id BIGINT,
    grade VARCHAR(2),

    PRIMARY KEY (student_id, course_id)
);
```

Neither `student_id` nor `course_id` must be unique by itself, but their combination must be unique.

Composite keys are commonly used in junction tables that represent many-to-many relationships.

### Interview Answer

A composite key is a key composed of multiple columns that together uniquely identify a row. It is commonly used in junction tables, such as a student-course relationship.

## 1.42 When Do We Use an OUTER JOIN in SQL?

An `OUTER JOIN` is used when unmatched rows should also be included in the result.

For example:

```
SELECT u.name, o.id
FROM users u
LEFT JOIN orders o
    ON u.id = o.user_id;
```

A `LEFT JOIN` returns all rows from the left table and matching rows from the right table.

If no matching row exists, the columns from the right table contain `NULL`.

```
INNER JOIN -> only matching rows
LEFT JOIN  -> all rows from the left table
RIGHT JOIN -> all rows from the right table
FULL JOIN  -> all rows from both tables
```

### Interview Answer

`OUTER JOINS` are used when we want to include unmatched rows in the result. For example, a `LEFT JOIN` can return all users even if some users have no orders.

## 1.43 What is a Database View and When Do We Use It?

A database `VIEW` is a virtual table based on a SQL query.

Views are commonly used to simplify complex queries, reuse query logic, restrict access to specific columns or rows, and provide abstraction over underlying tables.

A normal view generally stores the query definition rather than storing the query result itself.

## Interview Answer

A database view is a virtual table based on a SQL query. It is useful for simplifying complex queries, reusing query logic, restricting access to specific data, and hiding underlying table complexity.

### 1.44 What is the Difference Between a Process and a Thread?

A process is an independent running program with its own memory space.

A thread is a unit of execution inside a process.

Multiple threads in the same process share resources such as heap memory, while each thread has its own stack.

#### Process

```
|
+-- Shared Heap
|
+-- Thread 1 -> own stack
+-- Thread 2 -> own stack
+-- Thread 3 -> own stack
```

Processes are more isolated and generally more expensive to create, while threads are lighter and can communicate through shared memory.

However, shared memory can introduce concurrency problems such as race conditions.

## Interview Answer

A process is an independent running program with its own memory space, while a thread is a unit of execution inside a process. Threads in the same process share resources such as heap memory, but each thread has its own stack.

### 1.45 What is Multithreading?

Multithreading means executing multiple threads within the same process.

Threads share process resources such as heap memory, while each thread has its own stack.

Calling `start()` creates a new thread of execution, while calling `run()` directly is only a normal method call.

Multithreading can improve responsiveness and concurrency, but shared state must be handled carefully to avoid problems such as race conditions.

## Interview Answer

Multithreading is the execution of multiple threads within the same process. Threads share process resources but have separate stacks. It improves concurrency, but shared mutable state can introduce thread-safety problems.

## 1.46 What is Multiprocessing?

Multiprocessing means running multiple independent processes at the same time.

Each process has its own memory space and resources.

Process 1

-> Separate memory

Process 2

-> Separate memory

This is different from multithreading, where multiple threads run inside the same process and share memory.

Multiprocessing provides better isolation and can make effective use of multiple CPU cores.

However, communication between processes is generally more expensive and usually requires mechanisms such as pipes, sockets, shared memory, or message queues.

### Interview Answer

Multiprocessing means running multiple independent processes, each with its own memory space. It provides strong isolation and can use multiple CPU cores effectively, but inter-process communication is more expensive than communication between threads.

## 1.47 What is the Difference Between Multithreading and Multiprocessing?

Multithreading uses multiple threads inside the same process, while multiprocessing uses multiple independent processes.

Multithreading

-> One process

-> Multiple threads

-> Shared memory

-> Lower overhead

Multiprocessing

-> Multiple processes

-> Separate memory

-> Better isolation

-> Higher overhead

Threads can communicate easily through shared memory, but this may create race conditions and synchronization problems.

Processes are more isolated, but communication between them usually requires inter-process communication mechanisms.

### Interview Answer

Multithreading uses multiple threads inside the same process, so they share memory and are lightweight. Multiprocessing uses separate processes with independent memory spaces, providing better isolation but higher overhead.

## 1.48 What is a Race Condition?

A race condition occurs when multiple threads access and modify shared mutable data concurrently, causing the result to depend on the timing of the threads.

For example:

```
class Counter {  
  
    private int count = 0;  
  
    public void increment() {  
        count++;  
    }  
}
```

If multiple threads call `increment()` at the same time, some updates may be lost because `count++` is not an atomic operation.

Race conditions can be prevented using synchronization, locks, or atomic classes such as `AtomicInteger`.

### Interview Answer

A race condition occurs when multiple threads modify shared data concurrently and the result depends on the order or timing of execution. Proper synchronization or atomic operations can be used to prevent it.

## 1.49 What is Synchronization and Why is it Needed?

Synchronization controls access to shared mutable data when multiple threads are running concurrently.

For example:

```
class Counter {  
  
    private int count = 0;  
  
    public synchronized void increment() {  
        count++;  
    }  
}
```

The `synchronized` keyword ensures that only one thread at a time can execute the protected section for the same object.

Synchronization helps prevent race conditions and inconsistent data.

However, excessive synchronization can reduce performance because threads may need to wait for locks.

### Interview Answer

Synchronization controls concurrent access to shared mutable data. It protects critical sections so that only one thread at a time can execute them, helping prevent race conditions.

## 1.50 What is a Deadlock?

A deadlock occurs when two or more threads wait indefinitely for resources held by each other, so none of them can continue execution.

For example:

Thread 1:  
holds Lock A  
waits for Lock B

Thread 2:  
holds Lock B  
waits for Lock A

This creates a circular dependency.

One common way to reduce deadlock risk is to acquire locks in a consistent order.

### Interview Answer

A deadlock occurs when threads wait indefinitely for locks held by each other, preventing all of them from making progress.

## 1.51 What Does Thread-Safe Mean?

Thread-safe code behaves correctly when it is accessed by multiple threads concurrently.

For example, this is not thread-safe:

```
class Counter {  
    private int count = 0;  
  
    public void increment() {  
        count++;  
    }  
}
```

It can be made thread-safe using synchronization:

```
public synchronized void increment() {  
    count++;  
}
```

Other approaches include atomic classes, locks, and immutable objects.

### Interview Answer

Thread-safe code works correctly when multiple threads access it concurrently. Shared mutable state must be protected from race conditions and inconsistent updates.

## 1.52 What Are the Advantages of Stored Procedures?

A stored procedure is a reusable set of SQL statements stored and executed inside the database.

Main advantages include:

- Reusing database logic
- Reducing network communication
- Centralizing complex database operations
- Providing controlled access to data
- Potential performance benefits depending on the DBMS

**Interview Answer**

A stored procedure is a reusable set of SQL statements stored in the database. It can reduce network communication, centralize database logic, improve access control, and provide performance benefits in some cases.